Farmer-Herder Conflict Risk in Southern Chad — Technical Report

Final version of the product · BSc Software Engineering (Machine Learning), African Leadership University · Sougnabe · October 2026 · Repository: github.com/Sougnabe/chad-conflict-risk

1. Product overview

The product estimates, for each of the 26 departments of southern Chad and each quarter, the risk of violent farmer-herder conflict, to help humanitarian organisations, local authorities and mediation committees focus prevention. It consists of (i) a data pipeline that builds a validated conflict dataset from ACLED (1,593 events classified with a written codebook; blind validation, Cohen's kappa 0.83) and monthly satellite indicators from Google Earth Engine; (ii) a machine-learning layer that trains and evaluates risk models without look-ahead; and (iii) a bilingual (English/French) Streamlit dashboard that shows the outlook for the coming quarter, the factors behind each department's score, a replay of 2024–2025 and the research findings. The code is a Python package (*chadrisk*, 26 modules) with thin command-line scripts, 70 automated tests and a generated notebook.

2. Scope alignment and feature validation

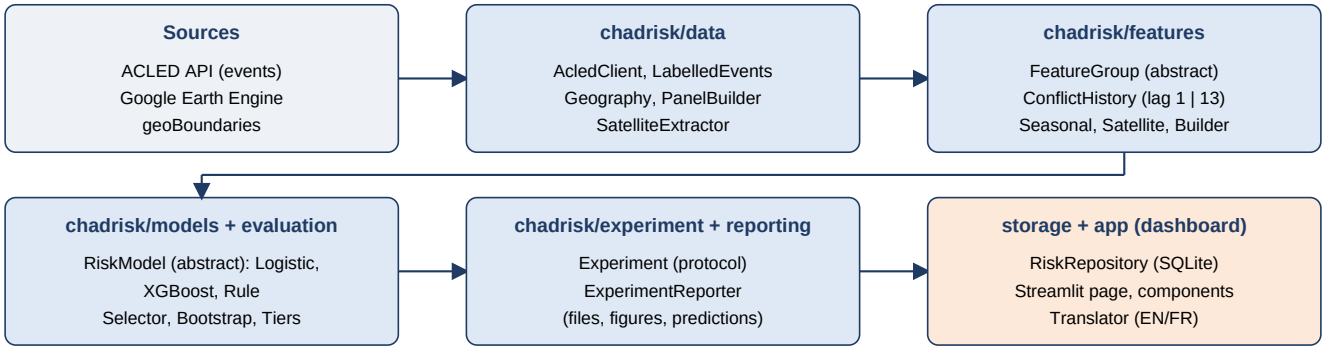
The table maps the features of the approved project brief (and the changes made since) to their status and evidence. Paths refer to the repository.

Planned feature	Intended purpose	Status	Evidence
Validated farmer-herder event dataset	ACLED has no farmer-herder category; a reliable target is needed	Completed	docs/labeling_codebook.md; data/processed/manual_labels.csv; kappa 0.83 on 60 blind-validated events
Department × month panel (26 departments)	Unit of analysis at the level where authorities act	Completed	chadrisk/data/panel.py, geography.py (spatial join, 0 events unassigned)
Satellite indicators (Earth Engine)	Pasture, rainfall and crop conditions, monthly	Completed	chadrisk/data/satellite.py; 26 depts × 105 months, incremental update
North–south link (herd departure zones)	Test whether northern pasture drives southern risk	Completed	north_ndvi_anom, north_rain_anom; feature sets C and OB
Crop-calendar timing	Detect standing crops vs harvest	Partial	Monthly crop-greenness anomaly and change used; annual MODIS phenology dropped (one value per year, too coarse for a monthly model)
Models with baseline and temporal validation	Answer the research question without look-ahead	Completed	chadrisk/experiment.py; reports/metrics_*.csv
Explanations (SHAP, risk factors)	Make scores interpretable	Completed	SHAP figures; per-department logistic contributions in the dashboard
Risk map dashboard	Present results to practitioners	Completed	app/; docs/screenshots/
Monthly forecast → quarterly outlook	Monthly risk proved unpredictable (no model beats chance)	Changed (justified)	Monthly PR-AUC 0.068 vs chance 0.038; quarterly results in Section 3
Operational model (added)	ACLED publishes with a 12-month delay; a real forecast must not rely on recent conflicts	Completed (added)	--mode operational; outlook for Oct–Dec 2026
Bilingual EN/FR interface (added)	Users in Chad work in French	Completed (added)	app/i18n.py (Translator)
Monthly update in one command (added)	Keep the outlook current	Completed (added)	scripts/update_outlook.py
Deployment	Shareable access for the demo and practitioners	Completed	Streamlit Community Cloud from app/app.py (link in README)
Practitioner feedback	External validation of usefulness	In progress	docs/practitioner_feedback_form.md sent; responses reported in the final report
Safer grazing-area map (optional)	Turn the warning into a recommendation	Not implemented	Deferred to future work: needs field validation of land use

3. Technical implementation

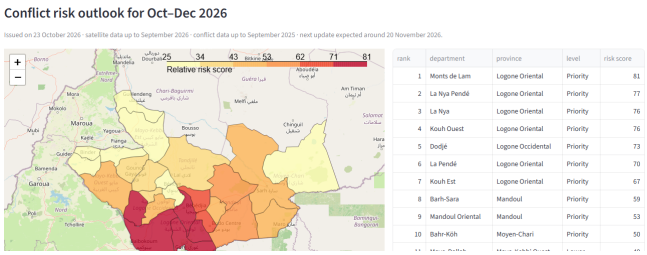
Backend–frontend interaction. The pipeline writes results to an SQLite database and to report files. The dashboard never touches SQL: it reads through *RiskRepository* (encapsulation), caches the parsed data once per session, and renders only the selected page. The database path is configurable (*CHADRISK_DB*); a missing database produces a clear message instead of a crash. **Data handling.** Every feature is lagged: research features use conflicts up to month t-1; operational features use conflicts up to t-13 (ACLED delay) and satellite data up to t-1. A leakage guard refuses any outcome column as a feature. Raw ACLED data and event notes are excluded from the repository (licence); only aggregated department-level data are stored. **Core logic.** An *Experiment* selects hyperparameters by rolling-origin validation on 2019–2023, evaluates once on 2024 – Sept 2025 (PR-AUC with month-block bootstrap intervals), computes priority tiers and a sensitivity analysis, refits on all data and forecasts the latest available period; an *ExperimentReporter* writes metrics, figures and predictions.

Architecture. A layered architecture in which each layer only depends on the layers before it; scripts only parse arguments and call the package.

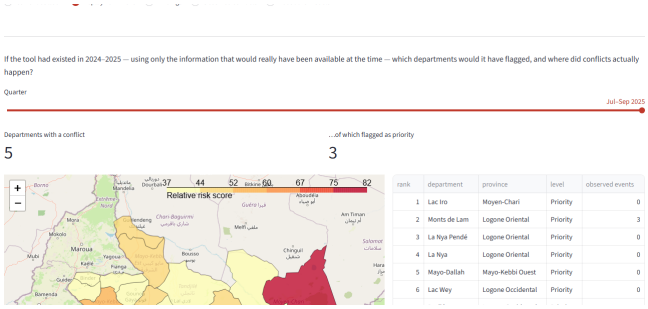


Quarterly test results (prevalence 0.11)	PR-AUC [95% CI]	ROC-AUC
Research — rule: conflicts in last 12 months	0.178	0.65
Research — history (logreg A)	0.210 [0.11–0.36]	0.65
Research — history + southern satellite (logreg B)	0.244 [0.12–0.39]	0.67
Operational — delayed history only (logreg OA)	0.110 [0.08–0.18]	0.46
Operational — delayed history + satellite (logreg OB, dashboard)	0.214 [0.11–0.41]	0.66

In the 2024–2025 replay, the 10 departments ranked as priority each quarter had a conflict in 20% of department-quarters, against 5% for the others. Intervals are wide (20 positive test quarters): the satellite contribution is consistent evidence, not proof.



Current outlook (Oct–Dec 2026): risk map, ranked departments and priority levels.



Replay 2024–2025: what the model flagged against the conflicts that happened.

4. Algorithms and technical problem-solving

Algorithm / custom logic	Why it is needed	Implementation and cost
Rolling-origin (expanding-window) model selection	Time-ordered data: random cross-validation would train on the future	RollingOriginSelector; candidate × fold pairs scored in parallel (joblib)
Month-block bootstrap	Departments in the same month are correlated; row resampling would understate uncertainty	MonthBlockBootstrap: 1,000 resamples of whole months
Neighbourhood graph with spatial index	Neighbouring conflicts are a risk factor	R-tree query (sindex) tests only candidate pairs instead of all n^2 pairs
Spatial join of events	ACLED and boundary names do not match	Point-in-polygon join on coordinates (0 unassigned events)
Months since last conflict	Recency of violence	Vectorised O(n): index of last positive value propagated with ffill (replaced a Python loop)
Delayed-history features	Respect real data availability	One class ConflictHistoryFeatures(shift) reused for t-1 and t-13; values masked past data end
Leakage guard and priority tiers	Prevent cheating; give scores a meaning	Set intersection with outcome columns; per-period ranking and conflict rate by band

Problems solved during development. (1) A model scored a perfect 1.0: a naming collision let outcome counts enter the features; fixed by renaming and a permanent leakage guard. (2) XGBoost results changed with the number of threads (order of floating-point sums); each model now uses one thread and parallelism is applied across candidates, which makes results identical on any machine. (3) ACLED's 12-month delay made a 'forecast' of a past month meaningless; an operational model was designed around real data availability. (4) Correlated history coefficients had opposite signs; the dashboard groups them so the explanation is coherent.

5. Testing strategies and results

Strategy	Why selected	How conducted	Results	Improvements made
Unit testing (48 tests)	Each class must be correct in isolation, incl. edge cases	pytest on small synthetic data: lags, neighbours, models, metrics, selection, repository, translations, invalid inputs	48/48 pass	Exposed the need for input validation (zero lag, single-class data, inverted dates, invalid labels)
Integration testing (14 tests)	Guarantee properties of the real outputs	Checks on the real panel, features and predictions: no look-ahead row by row, no leakage, valid predictions	14/14 pass	Found the out-of-period event (2 Oct 2025) and stale prediction rows
Functional UI testing with different inputs (8 tests)	Users click everywhere	Streamlit AppTest: every page in EN and FR, every replay quarter, every department, empty and narrow filters, missing database	8/8 pass	Found a crash when switching language on another page (page now stored as a number)
Performance testing	Field users have slow devices and networks	scripts/benchmark.py: time and peak RSS per stage, before vs after	See 6	Parallel selection, cached data, single-page rendering, lighter map
Compatibility testing	Users have different browsers and phones	scripts/browser_check.py (Playwright): 3 engines, desktop, mobile, low-end emulation	7/7 pass, 0 JS errors	Language switch moved out of the sidebar, which phones hide

6. Performance and compatibility

Measured on Windows-11-10.0.26300, Python 3.13.2, 4-core Intel CPU, 17 GB RAM. 'Before' is the same benchmark run on the version before the performance work (commit f201fc6).

Stage	Before (s)	After (s)	Speed-up	Peak RSS after (MB)
Experiment operational/quarter	25.532	19.542	×1.3	212.7
Experiment research/month	82.48	17.583	×4.7	214.4
Dashboard: cold start (first render)	7.771	1.544	×5.0	246.6
Dashboard: warm rerun (cached data)	1.199	0.129	×9.3	250.0
Dashboard: switch to French	1.396	0.17	×8.2	252.9
Map outlines sent to the browser	140 KB	51 KB	–64%	–

Browser / device (Playwright)	Content shown (s)	Map drawn (s)	Pages + FR switch	JS errors
Chromium desktop	0.7	4.2	ok	0
Firefox desktop	1.8	6.8	ok	0
WebKit desktop (Safari engine)	0.8	5.7	ok	0
Chromium mobile (Pixel 5)	0.4	4.5	ok	0
WebKit mobile (iPhone 13)	0.7	4.4	ok	0
Firefox small screen (390 px)	1.5	5.6	ok	0
Chromium low-end (CPU x4, Fast 3G)	14.3	32.9	ok	0

The low-end scenario (4× slower CPU, 'Fast 3G' network) remains usable but slow to draw the map (≈33 s); a lighter static map for low-bandwidth users is listed as future work. The training pipeline peaks at about 215 MB of memory.

7. Code quality and OOP practices

Principle	Where it is applied
Abstraction & polymorphism	RiskModel (fit, predict_risk, candidates, clone) implemented by LogisticRiskModel, XGBoostRiskModel and HistoryRuleModel; FeatureGroup implemented by the history, seasonal and satellite groups. Selection, evaluation and prediction work with any implementation.
Inheritance & reuse	Shared validation and cloning in the RiskModel base class; one ConflictHistoryFeatures class serves both research (lag 1) and operational (lag 13) features.
Encapsulation	RiskRepository hides SQL; AcledClient hides authentication; Translator hides languages; SatelliteExtractor hides Earth Engine.
Separation of concerns & modularity	Computation (Experiment) separated from output (ExperimentReporter); configuration in one module; dashboard split into page, components and translations; scripts are thin entry points.
Readability & maintainability	Docstrings on every module and class, consistent snake_case naming, pinned dependencies, deterministic results, 70 tests, meaningful commit history, README with structure and installation.

Limitations. Few positive cases (wide intervals); ACLED reporting bias and 12-month delay; herds inferred, not observed; the dashboard model (logistic OB) was fixed by design after the test results were known, which the model card states; practitioner feedback is still being collected.